CST2550 Pairly Dating App Project

Eduard Cojocaru

Aaron Pinto

Rajab Alasgarov

Alhussain Saiyed

Karish Shahi

Introduction

This project is all about the design and full stack development of Pairly. Pairly is a dating site that we worked on and it basically lets users make profiles, look at some potential matches, message other people using the site, and manage their account using settings. We as a group created this website as part of a software engineering group project using the Blazor structure, which uses C# for the backend, as it works well for web design. Our website contains many features like registering and logging into the website, editing profiles, swiping on people, matching with others, messaging, and using search tools. The main aim of the whole project was to make a functional, straight forward and easy to use system while following the AGILE framework like planning, designing, building, and testing. This report explains the main technical parts of the project. It starts with the design section, where it talks about the data structures and algorithms used and why they were chosen for the system. Then it all goes into the testing section, which explains how the main system was tested and the main test cases that were used by our group tester. Finally, it ends with a conclusion, where it sums up the work done, talks about any problems or limitations, and explains what could be improved in the future.

These are all the group members and their tasks; Eduard was managing this project and making sure that all tasks were completed on time. He organised team meetings, asked questions and assigned responsibilities. Aaron was responsible for the report, documenting all aspects of the project, system design explanations, and technical documentation such as the figma wire frames and diagrams. He ensured that all written content was clear and well structured. Aaron also helped track team progress and helped with some of the pages and front end.

Rajab contributed to the core development of the website using C# and SQL. He worked on implementing features such as user authentication, profile management, and system functionality based on the UML design. Alhussain contributed to the development process, focusing on implementing front-end and back-end features. He contributed to building interactive components such as swiping functionality, matching logic, and user interface behaviour. Karish was in charge of testing the application to ensure functionality, usability, and reliability. He identified bugs, tested different user scenarios, and gave feedback to improve system performance for a smoother user experience.

Design

Data Structure Selection and Justification

The Pairly system was designed using a relational data structure implemented through Entity Framework and a database context. Instead of storing data in a single linear collection, the system separates information into related entities such as Profile, Like, Match, and Message. This structure was selected because the application needs to manage several different types of linked data at the same time, including user profiles, interactions between users, confirmed matches, and communication history.

This approach is suitable for the system because dating applications depend heavily on relationships between records. For example, a like must link one user to another user, a match must connect two users once interest is mutual, and messages must be associated with a specific match. A relational structure makes this easier to organise and retrieve than storing all information in one general collection. It also improves maintainability, as each entity has a clear responsibility within the system.

The use of service classes such as matchservice also improves the design by separating business logic from the user interface. This makes the system easier to test, manage, and extend in future development. Now here are the algorithms that we designed for the web application.

Algorithm Design and Analysis

SWIPING FUNCTION

```
FUNCTION swipe(userId, targetUserId, isLike, isSuperLike):
    result.isMatch = false

    IF isLike is false:
        RETURN result

    IF like already exists from userId to targetUserId:
        RETURN result

    SAVE like(userId → targetUserId, superLike)
    mutualLike = check if targetUserId liked userId

    IF mutualLike exists:
        user1 = smaller(userId, targetUserId)
        user2 = larger(userId, targetUserId)

        IF active match does not exist:
            match = CREATE Match(user1, user2)
            SAVE match
            result.isMatch = true
            result.match = matched user profile
    RETURN result
```

GET MATCHES FUNCTION

```
FUNCTION getMatches(userId):
    matches = FIND active matches where user participates

    FOR each match:
        matchedUserId =
            IF user is user1 → user2
            ELSE user1
        profile = GET profile of matched user
        lastMessage = GET most recent message
        unreadCount = COUNT unread messages from other user
        ADD match data to result list
    RETURN result list
```

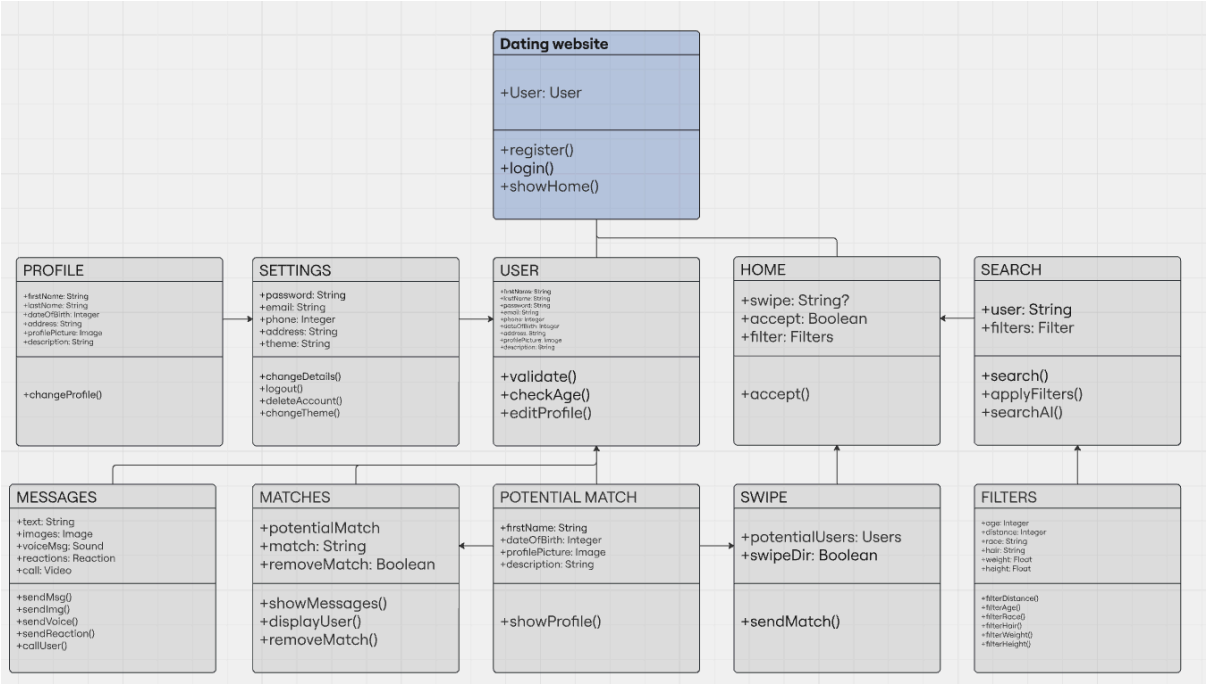
GET PROFILES FOR HOME/SEARCH PAGE FUNCTION

```
FUNCTION getDiscoveryProfiles(userId, filters):
    userProfile = GET current user profile
    likedUsers = users already liked by current user
    matchedUsers = users already matched with current user
    profiles = GET all profiles except current user
    REMOVE likedUsers from profiles
    REMOVE matchedUsers from profiles

    APPLY filters:
        gender
        search term
        hair color
        eye color
        body type
        ethnicity
        smoking
        drinking

    RANDOMIZE order
    LIMIT results to filter.count OR 20
    RETURN profiles
```

Here is our UML diagram as well which displays how data is stored inside our web application.



Swipe and Match Detection Algorithm

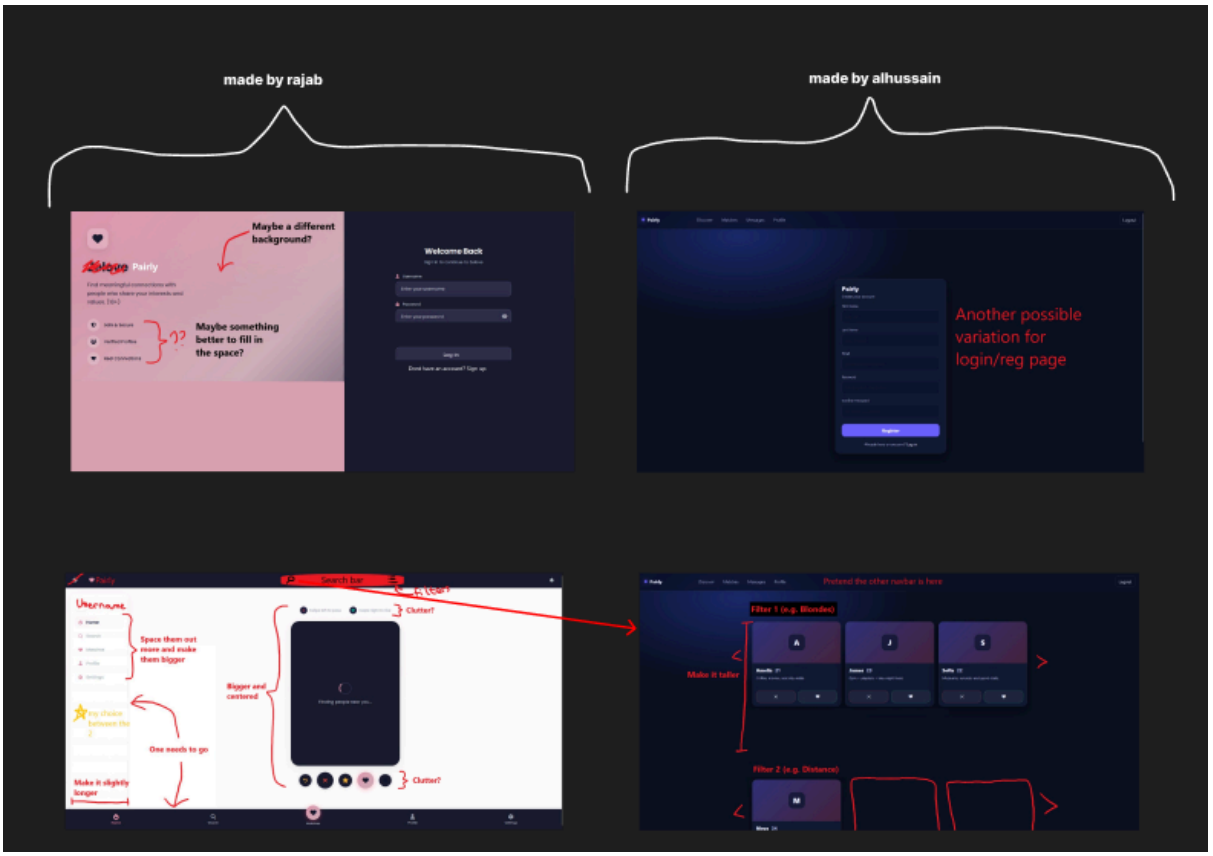
The swipe process begins by checking whether the swipe is a positive interaction. If the user does not like the profile, the process ends immediately. If it is a like, the system checks whether the same like has already been stored. This prevents duplicate records. If no existing like is found, a new like is added. The system then checks whether the target user has already liked the current user. If so, a new match is created.

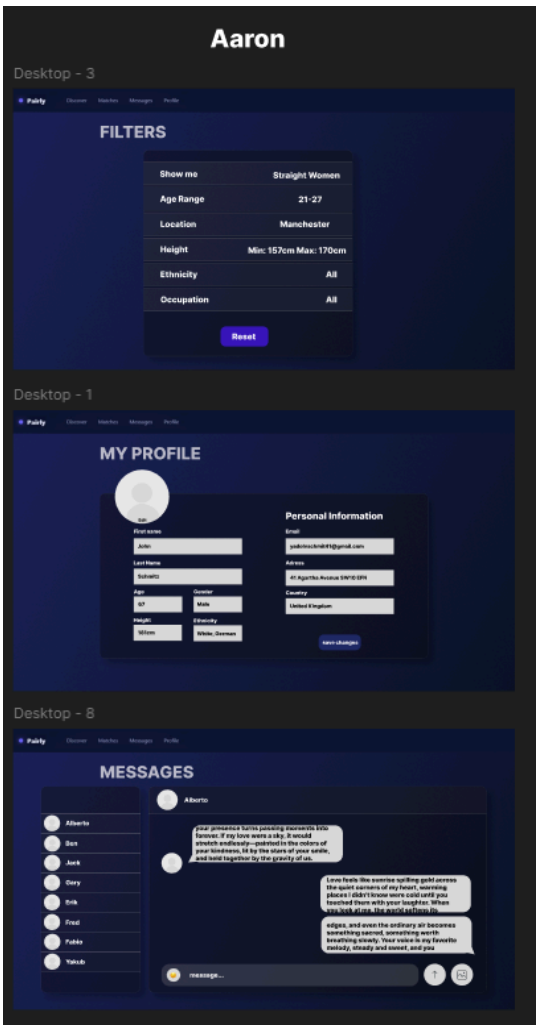
Time complexity

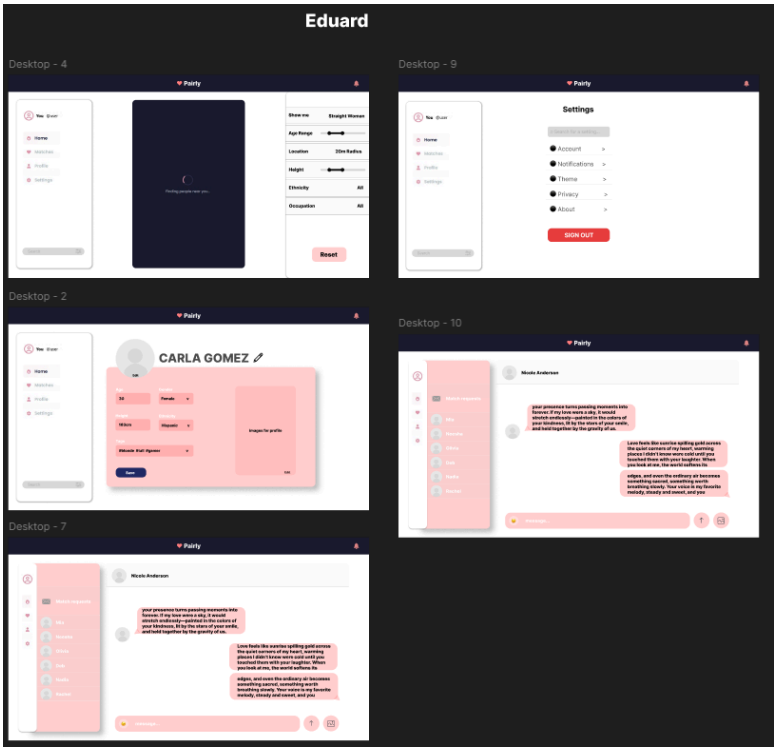
Most of the operations in this system have time complexity of O(n) because of searching through stored records, while other operations such as adding new records are typically O(1). Retrieval of matches and messages operates in O(m) and O(k) respectively, depending on the number of matches or messages processed.

Figma Designs – Dating Website Wireframe

The next images consist of the different wireframes that we worked on for the project. Which show our plans on how we wanted the website to look like and also different modifications to existing designs that were created.







Testing

The testing of the Pairly system was carried out by our tester Karish using unit testing to make sure that core functionality of the website actually ran correctly. The NUnit testing framework was very much used to make and execute automatic test cases, allowing the individual components of the system to be tested in isolation. To make sure consistent and very reliable testing, a database provided by Entity Framework Core was also used. This actually allowed the whole system to simulate the database operations without needing a real SQL server, making sure that tests could be carried out quickly and on their own. Each test case used a fresh instance of the database to avoid interference between tests. The testing focused on key components of the system, including authentication, profile management, matching functionality, and messaging. These areas were selected as they represent the core features of the application and are critical to ensuring correct system behaviour.

Test Case	Description	Expected Result	Outcome
Register user with valid data	User provides correct registration details	User is successfully registered and token is generated	pass
Register with duplicate username	User attempts to register with an existing username	System prevents registration and returns error	pass
Login with valid details	User logs in with correct username and password	User is authenticated and token is returned	pass
Login with invalid password	User enters incorrect password	Login fails and returns null	pass
Swipe mutual match	Two users like each other	System creates a match and returns success	pass
Get likes received	User checks who liked them	System returns list of users who liked them	pass
Get matches	User retrieves matches	System returns list of active matches	pass
Send message	User sends message in valid match	Message is stored and returned correctly	pass
Retrieve message	User opens chat	Messages are returned and unread messages are marked as read	pass
Delete message	User deletes their message	Message is removed successfully	pass
Update profile	User updates profile details	Profile is updated correctly	pass
Discover profiles	User searches for matches	System returns filtered profiles	pass
Login with valid credentials (failed case)	User logs in with correct username and password but test data may not match hashing or database setup	User should be authenticated and return a valid response	fail

Conclusion

In conclusion, the code clearly shows the full functionality of our dating website system and how everything works behind the scenes. It highlights how different features such as swiping, matching, and messaging are actually implemented and connected together. The system is well organised and follows the structure from the UML design, making it easier to understand and develop further. Karish also tested the application to make sure everything works properly, checking different features and finding any issues that needed fixing. Overall, it provides a strong working foundation for the application and shows that the system is functional, tested, and ready for further improvements moving forward.

Overall, the Pairly dating app was designed and developed successfully and shows the main software engineering ideas being used. The system includes features like user registration, profile management, swiping, matching, and messaging. The data was organised in a structured way so it can be managed properly, and things like swiping and matching were handled correctly by the system. Testing was also done to make sure the main parts of the system actually work as expected.

However, there were some limitations in the project. One main issue is that a lot of things in the system require searching through data, which can get slower when there are more users and more activity. This was mainly because the system was kept simple and there was not enough time to fully improve it. Another issue is that some features were not fully improved or extended, as the main focus was just getting the core features working properly instead of making everything faster or more advanced.

From a reflection point of view, the development showed that planning from the start is really important. Some time was spent on things that were not fully needed for the marking criteria, which could have been avoided if everything was planned better. Also the writing in the report could have been more consistent, which would have made the overall work better.

If the project was done again, there would be a few improvements. Better data handling or optimisation could be used to make things faster, especially for searching and matching. More focus would be put on following the marking criteria from the start so everything is clear and relevant. Also more testing would be done, including different edge cases, to make sure the system is more reliable.

Overall, the project gave good experience in building a working system and showed how software development works, while also showing what could be improved next time.

References

- Cormen, T. (2009). *Introduction to Algorithms*. MIT Press.
- Microsoft (2024). *Entity Framework Core Documentation*. Available at: https://learn.microsoft.com/
- Microsoft (2024). *Blazor Documentation*. Available at: https://learn.microsoft.com/